INF1771 - INTELIGÊNCIA ARTIFICIAL

TRABALHO 2 – AGENTE LÓGICO – PITFALL!

Nosso agente deve planejar o caminho do personagem "Pitfall Harry" por uma floresta em forma de labirinto, buscando recuperar tesouros com eficiência e sobreviver aos perigos. O ambiente contém inimigos, poços e locais de teletransporte, e o agente deverá decidir suas ações com base nas percepções disponíveis.

No caminho, Pitfall deverá superar riscos, incluindo poços que provocam sua morte imediata. Como o mapa real é desconhecido para o agente, será necessário inferir quais posições podem ser seguras antes de explorá-las.

Descrição:

O segundo trabalho é baseado no exemplo do "Mundo do Wumpus". O agente explora um labirinto sem conhecer previamente a localização de seus elementos, utilizando percepções locais e de salas adjacentes para construir conhecimento, inferir riscos e planejar suas ações. O objetivo é coletar ouro, preservar a energia e obter a maior pontuação possível.

O Trabalho 2 consiste em implementar um agente lógico autônomo a partir do projeto-base em SWI-Prolog e Python. A representação do conhecimento, as regras de inferência e a escolha de ações deverão ser implementadas em Prolog. A interface em Python poderá exibir a execução, a pontuação e, quando apropriado, uma representação do conhecimento inferido pelo agente.

Características

- O labirinto deve ser representado por uma matriz 12 x 12.
- O agente sempre inicia na posição [1, 1] do labirinto.
- A posição [1,1] é a origem do agente. O retorno a essa posição pode ser adotado como objetivo adicional de saída, mas a condição de vitória por retorno à origem não está implementada no código-base e, caso seja exigida, deverá ser acrescentada.
- O ambiente possui inimigos de dois tipos, além de casas de teletransporte: inimigo pequeno (d) causa dano aleatório de 25 a 50 pontos de energia; inimigo grande (D)

causa dano aleatório de 50 a 80 pontos; teletransportadores (T) deslocam o agente para outra posição aleatória.

- O Agente terá 100 pontos de energia inicial.
- Existem 3 itens de recuperação de energia (U), cada um restaurando 50 pontos, sem ultrapassar o máximo de 100.
- O agente pode executar as seguintes ações:
    - Andar;
    - Virar_a_direita (rotação de 90° no sentido horário);
    - Virar_a_esquerda (rotação de 90° no sentido anti-horário);
    - Pegar – Para recolher ouro ou item de recuperação presente na posição atual;
- As ações andar e virar custam 1 ponto de pontuação; a ação pegar custa 5 pontos. Os demais eventos seguem as regras implementadas no código-base:
    - Pegar ouro (O) = +500 pontos, além do custo de 5 pontos da ação pegar (saldo líquido +495);
    - Cair em um poço (P) = -1000 pontos e energia reduzida a zero;
    - Os inimigos reduzem a energia do agente conforme seu tipo; o código-base não aplica automaticamente uma penalidade adicional de -1000 pontos pela morte;
    - Receber dano de inimigos reduz a energia, sem descontar diretamente o mesmo valor da pontuação.
- O agente possui os seguintes sensores:
    - Em salas adjacentes a inimigos, exceto diagonais, o agente ouve passos (palmas é reservado aos teletransportadores);
    - Em salas adjacentes a um poço/obstáculo, exceto diagonal, o agente sente uma brisa;
    - Em salas adjacentes a um teletransportador, exceto diagonais, o agente percebe palmas;
    - Na posição que contém ouro, o agente percebe brilho; na posição com item de recuperação, percebe reflexo;

    - O limite do labirinto funciona como parede: uma tentativa de deslocamento inválida falha no código-base; não existe atualmente uma percepção específica de impacto.
- O labirinto possui os seguintes elementos:
    - 2 inimigos pequenos (d), com dano aleatório de 25 a 50;
    - 2 inimigos grandes (D), com dano aleatório de 50 a 80;
    - 8 Poços/Obstáculos;
    - 3 Pedras de ouro;
    - 4 teletransportadores (T), além de 3 itens de recuperação de energia (U);
- Os mapas são definidos em arquivos Prolog fornecidos com o projeto (incluindo diferentes níveis de dificuldade). A distribuição dos elementos é fixa em cada arquivo; o código-base não sorteia um novo mapa completo a cada execução.
- O agente não pode consultar diretamente a representação completa do mapa para decidir suas ações. Suas decisões devem utilizar somente percepções, memória de posições visitadas e inferências logicamente justificadas.
- A execução termina quando o agente morre ou quando for atingida uma condição de encerramento definida para o trabalho. O código-base não implementa, por padrão, a vitória ao retornar à posição [1,1].
- Ao entrar em uma sala onde existe um teletransporter, o agente “acorda” em um lugar aleatório do labirinto, podendo ser um local seguro, um poço, a sala de um inimigo ou a sala onde existe outro teletranporter. Ou seja, o local onde o agente será teleportado deve ser sorteado.
- Nos slides das aulas anteriores existem vários exemplos do mundo de Wumpus, inclusive algumas regras em lógica de primeira ordem que podem ser traduzidas para Prolog.

Requisitos:

- O programa deve ser implementado em SWI-Prolog + Python (utilizar o código exemplo).

- O Prolog deve representar o conhecimento do agente, realizar inferências e decidir as ações. A interface visual e os controles permanecem em Python; algoritmos de busca, como A*, podem ser implementados em Python, desde que planejem exclusivamente sobre o mapa conhecido ou inferido, sem acesso indevido ao mapa real.
- A função executa_acao/1 fornecida no projeto-base escolhe ações aleatoriamente. O grupo deverá substituí-la por uma estratégia fundamentada em regras lógicas, memória de visitas, avaliação de risco e planejamento sobre informações conhecidas ou inferidas.
- Devem ser realizados testes nos mapas fornecidos, registrando pelo menos pontuação, quantidade de ouro coletado, sobrevivência e número de ações. O mapa real exibido para depuração não pode orientar as decisões do agente.
- O programa também deve exibir a pontuação do agente enquanto ele executa as ações. Assim como a pontuação final. (já implementado na interface exemplo)
- Os arquivos de mapas fornecidos pelo projeto devem poder ser carregados para testar o agente em cenários distintos, sem inserir manualmente a matriz completa na base de conhecimento do agente.

Forma de Avaliação:

Será avaliado se:

(1) O agente construiu e atualizou corretamente sua base de conhecimento a partir das percepções, sem consultar o mapa real;

(2) As regras lógicas, a seleção de ações e o planejamento foram implementados corretamente e avaliados em diferentes mapas;

(3) O código foi organizado, integrado à interface e acompanhado de registros de testes (pontuação, ouro coletado, sobrevivência e ações executadas);

(4) O trabalho foi apresentado em vídeo (de até 10 mins), com explicação da estratégia, demonstração e discussão das limitações;

Recomendações Finais:

Muito do trabalho consiste em representar fatos, formular inferências, definir o ciclo de percepção–decisão–ação e integrá-lo ao programa em Prolog. A função executa_acao/1 do projeto-base realiza escolhas aleatórias e deverá ser

substituída por uma política lógica. Testem a solução em diferentes mapas e tragam suas dúvidas durante o desenvolvimento.